LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

MODUL IV DAN V

THE CORE OF OOP

ENCAPSULATION

Disusun Oleh:

Intan Rhevita Arselina Suryana (105224009)

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS SAINS DAN KOMPUTER

UNIVERSITAS PERTAMINA

2026

I. PENDAHULUAN

1.1 Latar Belakang

Pemrograman Berorientasi Objek (*Object-Oriented Programming* atau OOP) merupakan paradigma pemrograman yang berfokus pada konsep objek dan kelas. Paradigma ini banyak digunakan dalam bahasa pemrograman modern seperti Java karena kemampuannya dalam membangun sistem yang terstruktur dan mudah dikembangkan.

Inti dari OOP terletak pada empat pilar utama, yaitu *encapsulation*, *inheritance*, *polymorphism*, dan *abstraction*. Keempat konsep tersebut saling berkaitan dan menjadi dasar dalam membangun sistem yang modular serta mudah untuk dikembangkan (*scalable*).

Salah satu konsep utama dalam OOP adalah *encapsulation* atau enkapsulasi. Enkapsulasi merupakan prinsip yang bertujuan untuk membungkus data dan metode ke dalam satu kesatuan berupa kelas, serta membatasi akses langsung terhadap data tersebut. Dengan demikian, data hanya dapat diakses dan dimodifikasi melalui metode tertentu yang telah ditentukan.

Konsep ini memungkinkan pengembang untuk mengontrol penggunaan data, menjaga integritas, serta meningkatkan keamanan data dalam suatu program. Selain itu, detail implementasi internal suatu kelas dapat disembunyikan dari pengguna luar (*data hiding*), sehingga perubahan pada bagian internal tidak memengaruhi bagian lain dari sistem.

Penerapan enkapsulasi juga memberikan berbagai manfaat, seperti meningkatkan keterbacaan kode, mempermudah proses pemeliharaan, serta memungkinkan perubahan dilakukan tanpa mengganggu bagian lain dari program. Dengan demikian, enkapsulasi berperan penting dalam menciptakan perangkat lunak yang lebih terstruktur, aman, dan fleksibel.

Oleh karena itu, penerapan konsep inti OOP, khususnya enkapsulasi, menjadi sangat penting dalam pengembangan perangkat lunak modern karena dapat meningkatkan kualitas kode, mempermudah proses debugging, serta mendukung prinsip *reusability* dan *maintainability*.

II. VARIABEL

No	Nama Variabel	Tipe Data	Fungsi
1	nomorKamar	String	Untuk menyimpan nomor kamar
2	tipeKamar	string	Untuk menyimpan tpe kamar
3	kapasitasMaks	Int	Untuk menyimpan kapasitas maksimal pelanggan hotel yang menyewa kamar hotel
4	hargaPerMalam	double	Untuk menyimpan harga kamar permalamnya, menggunakan Boolean agar uang bisa lebih akurat dihitungnya
5	isTersedia	Boolean	Untuk menyimpan apakah apakah kamar tersedia atau tidak
6	jumlahTamu	Int	Untuk menyimpan jumlah tamu yang akan menempati kamar
7	jumlahMalam	Int	Untuk menyimpan jumlah malam tamu menginap
8	kodeVoucher	String	Untuk menyimpan kode voucher

III. DOKUMENTASI DAN PEMBAHASAN CODE

```java
public class KamarHotel {
    private String nomorKamar;
    private String tipeKamar;
    private int kapasitasMaks;
    private double hargaPerMalam;
    private boolean isTersedia;
```

Pertama ada bagian public class yang merupakan blueprint untuk objek KamarHotel. Blueprint tersebut diisi dengan 5 parameter dan memakai private karena pihak hotel meminta semua atribut tidak dapat diakses langsung dari luar kelas. Pihak hotel juga meminta setaip kamar memiliki atribut krusial nomorKamar (String), tipeKamar (String), kapasitasMaksimal (int), hargaPerMalam (double), dan isTersedia (boolean).

```java
    public KamarHotel (String nomorKamar, String tipeKamar, int kapasitasMaks){
        this.nomorKamar = nomorKamar;
        this.tipeKamar = tipeKamar;
        this.kapasitasMaks = kapasitasMaks;
        this.hargaPerMalam = 0;
        this.isTersedia = true;
    }

    public KamarHotel (String nomorKamar, String tipeKamar, int kapasitasMaks, double hargaPerMal
        this.nomorKamar = nomorKamar;
        this.tipeKamar = tipeKamar;
        this.kapasitasMaks = kapasitasMaks;
        this.hargaPerMalam = hargaPerMalam;
        this.isTersedia = true;
    }
```

Pihak hotel ingin isi constructor pertama untuk objek KamarHotel adalah nomorKamar, tipeKamar, dan kapasitasMaksimal lalu sistem akan otomatis mengatur harga menjadi 0 dan status isTersedia menjadi true. This digunakan untuk mereferensi ke objek yang di tuju. This.nomorKamar dan lainnya itu adalah milik hotel saat ini dan nomorKamar adalah data yang dikasi user. Sedangkan constructor kedua melayani pendaftaran lengkap dengan menerima keempat parameter utama (nomor, tipe, kapasitas, dan harga), dengan status ketersediaan yang juga langsung diatur true secara bawaan.

```java
public String getnomorKamar(){
 return nomorKamar;
}

public void setnomorKamar(String nomorKamar){
 this.nomorKamar = nomorKamar;
}

public String gettipeKamar(){
 return tipeKamar;
}

 public int getkapasitasMaks(){
   return kapasitasMaks;
 }
```

Gunakan getter untuk mengambil nilai dan setter untuk mengubah nilai. Getter/setter ini juga merupakan gerbang control data dalam object. Buat getter dan setter di semua variable.

```java
public void settipeKamar(String tipeKamar){
if (tipeKamar.equalsIgnoreCase(anotherString: "Reguler") || tipeKamar.equalsIgnoreCase(a
   this.tipeKamar = tipeKamar;
   } else {
     System.out.println(x: "Tipe kamar tidak valid! kami ubah ke Reguler.");
     this.tipeKamar = "Reguler";
     }
}

public double gethargaPerMalam() {
  return hargaPerMalam;
}

public void sethargaPerMalam(double hargaPerMalam) {
  if (hargaPerMalam < 50000) {
      System.out.println(x: "Harga terlalu murah! Diubah ke 50000.");
      this.hargaPerMalam = 50000;
     } else {
       this.hargaPerMalam = hargaPerMalam;
       }
}
```

Pada settipeKamar ini system ingin hanya menerima input "Reguler", "Premium", atau "Suite" dan menggunakan equalsIgnoreCase karena system tidak ingin mempedulikan huruf besar atau kecil agar mengurangi terjadinya kesalahan. Kalo pelanggan memilih pilihan yang diluar itu maka akan keluar pesan “Tipe kamar tidak valid! Kami ubah ke regular.” Karena tipe data yang dimasukkan tidak valid jadi akan di ubah paksa ke tipe kamar regular.

Pada sethargaPerMalam kalua pelanggan memasukkan angka di bawah 50.000 maka akan keluar pesan peringatan dan diubah paksa menjadi 50.000.

```java
    public boolean isTersedia() {
      return isTersedia;
    }

    public void pesanKamar() {
      if (isTersedia) {
          isTersedia = false;
        } else {
          System.out.println(x: "Kamar sudah terisi!");
          }
    }

    public void pesanKamar(int jumlahTamu) {
      if (!isTersedia) {
          System.out.println(x: "Kamar sudah terisi!");
       } else if (jumlahTamu > kapasitasMaks) {
          System.out.println(x: "Jumlah tamu melebihi kapasitas!");
          } else {
            isTersedia = false;
            }
    }

```

Pada metode pesanKamar jika kamar tersedia maka status akan diubah menjadi false dan jika kamar terisi maka akan keluar pesan bahwa kamar sudah terisi. Kemudian pada metode pesanKamar tidak hanya mengecek ketersediaan kamar saja tapi juga memvalidasi apakah jumlah tamu tidak melebihi kapasitas maksimal. Jika pesan ga berlebih maka sistem wajib menolak pesanan dan akan keluar pesan bahwa jumlah tamu melebihi kapasitas lalu status kamar akan tetap menjadi true.

```java
  public void batalPesan() {
      isTersedia = true;
  }

  public double hitungTotalBayar(int jumlahMalam) {
   return jumlahMalam * hargaPerMalam;
  }

  public double hitungTotalBayar(int jumlahMalam, String kodeVoucher) {
   double total = jumlahMalam * hargaPerMalam;

  if (kodeVoucher.equalsIgnoreCase(anotherString: "PROMO") && jumlahMalam >= 3) {
      System.out.println(x: "Voucher berhasil digunakan! Diskon 20%");
      return total * 0.8;
    } else {
      System.out.println(x: "Voucher gagal digunakan!");
      return total;
      }
  }
}

```

Sediakan metode batalPesan untuk mengembalikan kamar menjadi status tersedia. Gunakan metode hitungTotalBayar(int jumlahMalam) yang mengembalikan nilai total tagihan harga standar. Hitung nilai total bayar dengan mengkalikan jumlah malam x harga per malam. Lalu hitung lagi dengan menggunakan metode hitungTotalBayar namun tambahkan parameter kodevoucher dan tambahkan syarat jika kode voucher “PROMO” dimasukkan dan tamu yang menginap minimal 3 maka mereka akan mendapat diskon 20%. Jika syarat minimum malam tidak

terpenuhi atau kodenya salah, sistem harus mengembalikan nilai tagihan harga normal beserta pesan penolakan penggunaan voucher.

```
public class Main {
    Run | Debug
    public static void main(String[] args){
    KamarHotel kamar1 = new KamarHotel(nomorKamar: "3016", tipeKamar: "Standard", kapasitasMaks
        kamar1.settipeKamar(tipeKamar: "Penthouse");
        kamar1.sethargaPerMalam(-40000000);

    KamarHotel kamar2 = new KamarHotel(nomorKamar: "2918", tipeKamar: "Suite", kapasitasMaks: 8

        kamar2.pesanKamar(jumlahTamu: 4);
        kamar2.pesanKamar(jumlahTamu: 2);
        kamar2.pesanKamar();

    System.out.println(x: "Tagihan Kamar 1 :");
        double total1 = kamar1.hitungTotalBayar(jumlahMalam: 2, kodeVoucher: "PROMO");
        System.out.println("Total bayar: " + total1);

        System.out.println(x: "Tagihan Kamar 2 :");
        double total2 = kamar2.hitungTotalBayar(jumlahMalam: 4, kodeVoucher: "PROMO");
        System.out.println("Total bayar: " + total2);
```

Bagian paling atas yaitu public class main dan public static merupakan titik awal eksekusi program dalam bahasa Java. Method main berfungsi sebagai tempat pertama kali program dijalankan. Lalu lakukan pembuatan objek kamar1 dari class KamarHotel. Nilai seperti nomor kamar, tipe kamar, kapasitas, dan atribut lainnya diinisialisasi melalui constructor. Ubah tipe kamar1 menjadi penthouse dan ubah harga permalamnya menjadi -40000000. Lalu buat lagi objek kamar2 dengan atribut yang berbeda. Method pesankamar digunakan untuk jumlah pelanggan yang memesan kamar. Untuk pesanKamar yang terakhir isinya kosong untuk menunjukkan method overloading dalam OOP. Bagian tagihan kamar digunakna untuk menghitung dan enampilkan total biaya sewa kamar dengan isi jumlah malam dan menggunakan kode voucher PROMO. Hasil perhitungan disimpan dalam variabel total1 dan ditampilkan ke layar.

IV. KESIMPULAN

Kesimpulan dari praktikum ini adalah saya menjadi lebih memahami konsep dasar dalam pemrograman berorientasi objek, khususnya mengenai encapsulation. Melalui pembuatan program KamarHotel, saya belajar bagaimana cara membungkus data dan method dalam satu class serta membatasi akses langsung menggunakan modifier private.

Selain itu, penggunaan getter dan setter membantu dalam mengontrol data agar tetap valid dan tidak sembarangan diubah dari luar class. Saya juga memahami bagaimana constructor digunakan untuk menginisialisasi objek, serta bagaimana method dapat digunakan untuk menjalankan logika seperti pemesanan kamar dan perhitungan total biaya.

Dengan adanya penerapan konsep-konsep tersebut, program menjadi lebih terstruktur, mudah dipahami, dan lebih aman dari kesalahan penggunaan data. Praktikum ini membantu saya memahami bahwa dalam OOP tidak hanya sekadar membuat program berjalan, tetapi juga bagaimana menyusun kode yang rapi, terkontrol, dan mudah dikembangkan di masa depan.

V. DAFTAR PUSTAKA

https://codingstudio.id/blog/enkapsulasi-adalah/

https://dibimbing.id/blog/detail/apa-itu-enkapsulasi-definisi-manfaat-teknik-serta-contoh

https://dte.telkomuniversity.ac.id/pemrograman-berorientasi-objek-oop-konsep-implementasi-dan-contoh/